

ParkSim — Projeto de Banco de Dados

Estacionamento com simulador web, banco relacional em nuvem (PostgreSQL/Supabase)
e dados persistidos por conta (Auth + Row Level Security)

GRUPO 5

João Vitor Alves Rodrigues — RA: 32513480

Rafael Luiz Ferreira de Souza — RA: 32511503

Pietro Cardoso de Oliveira — RA: 32515280

Gustavo Henrique Ramos — RA: 325141430

Pedro Henrique Martins — RA: 325130235

Professores: Alexandre e Silvia

Disciplina: Banco de Dados

Setembro / 2026

lebot007.github.io/a3controlecarros

Estrutura da apresentação

- 1. Levantamento de requisitos e regras de negócio
- 2. Modelagem conceitual — Diagrama Entidade-Relacionamento (DER)
- 3. Modelagem lógica e normalização (até 3FN) + integridade referencial
- 4. Modelagem física e linguagem SQL — DDL, DML e DQL executáveis
- 5. Aplicação web, demonstração e persistência dos dados por conta
- 6. Conclusão e conceitos de Banco de Dados aplicados

Contexto da organização e problema identificado

- Organização: ParkSim — estacionamento fictício com 8 vagas físicas e 4 tipos de tarifa (Comum, Coberta, PCD e Idoso).
- Problema: controle de entrada/saída em papel e planilhas isoladas → duplicidade de dados, dificuldade de acesso e inconsistências.
- Solução: banco de dados relacional real (PostgreSQL via Supabase, em nuvem) integrado a uma aplicação web que simula e registra o pátio em tempo real.
- Entregas: documento de projeto (modelagens + diagramas), script .sql executável (DDL + DML + DQL) e site publicado com painel administrativo.

Atividades contempladas pelo projeto

- Registro de entrada de veículos com ocupação de vaga e atendente responsável;
- Registro de saída com cálculo automático de tempo e valor cobrado (regra dentro do banco, via TRIGGER);
- Cadastro de clientes e de seus veículos (1 cliente : N veículos);
- Gestão das vagas físicas e das tarifas por hora de cada tipo de vaga;
- Painel administrativo com relatório em tempo real (SELECT com JOINS) e exportação em PDF;
- Controle de acesso: cada usuário autenticado vê e gerencia apenas os dados da sua conta (Auth + RLS).

Informações que precisam ser armazenadas

- Clientes: nome, CPF, telefone e e-mail;
- Veículos: placa, cor, modelo e o cliente proprietário;
- Funcionários: nome, cargo e CPF (atendentes que registram os movimentos);
- Tipos de vaga: descrição e valor da tarifa por hora;
- Vagas: número, tipo da vaga e situação (ativa/inativa);
- Movimentações: veículo, vaga, funcionário, hora de entrada, hora de saída, tempo em minutos, valor cobrado, status, sessão e conta (user_id).

Necessidades de consulta e geração de relatórios

- Relatório completo das movimentações da conta, com JOIN entre as 6 tabelas (base do painel e do PDF);
- Faturamento total por cliente no último mês — exemplo citado no enunciado do desafio;
- Vaga mais utilizada (JOIN + COUNT + GROUP BY);
- Tempo médio de permanência por tipo de vaga (AVG);
- Funcionário que mais registrou movimentações (COUNT);
- Veículos atualmente estacionados, com dados do cliente e da vaga.

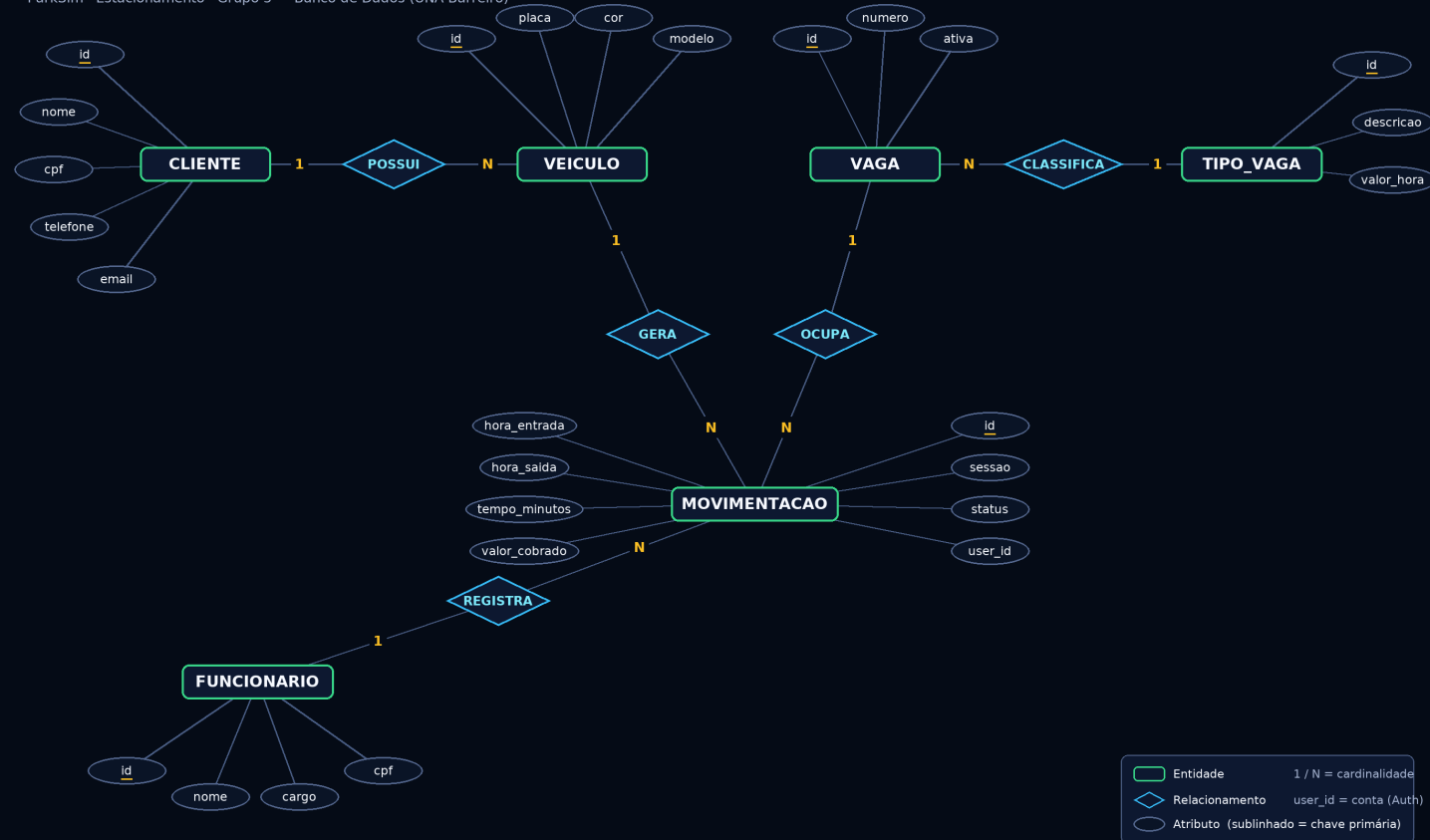
Regras de negócio que orientam o modelo

- RN01 — Um cliente pode possuir vários veículos; cada veículo pertence a um único cliente (1:N).
- RN02 — Cada movimentação envolve exatamente 1 veículo, 1 vaga e 1 funcionário.
- RN03 — Um veículo pode gerar várias movimentações ao longo do tempo (histórico).
- RN04 — A tarifa por hora é do TIPO da vaga, não da vaga (evita redundância → 3FN).
- RN05 — Na saída, o banco calcula tempo_minutos e valor_cobrado (TRIGGER trg_calcular_saida).
- RN06 — Saída somente após a entrada (CHECK) e status limitado a ESTACIONADO ou SAIU.
- RN07 — Vagas PCD e Idoso possuem tarifa social (valor_hora = 0,00).
- RN08 — Toda movimentação pertence à conta logada (user_id); visitante não grava dados.
- RN09 — Excluir um veículo exclui suas movimentações (CASCADE); vagas e funcionários são protegidos (RESTRICT).

Diagrama Entidade-Relacionamento (DER)

Modelagem Conceitual — Diagrama Entidade-Relacionamento (DER)

ParkSim · Estacionamento · Grupo 5 — Banco de Dados (UNA Barreiro)



Notação de Chen: 6 entidades, 5 relacionamentos 1:N, atributos com chaves primárias sublinhadas e user_id vinculando os dados à conta autenticada.

Entidades, chaves e cardinalidades

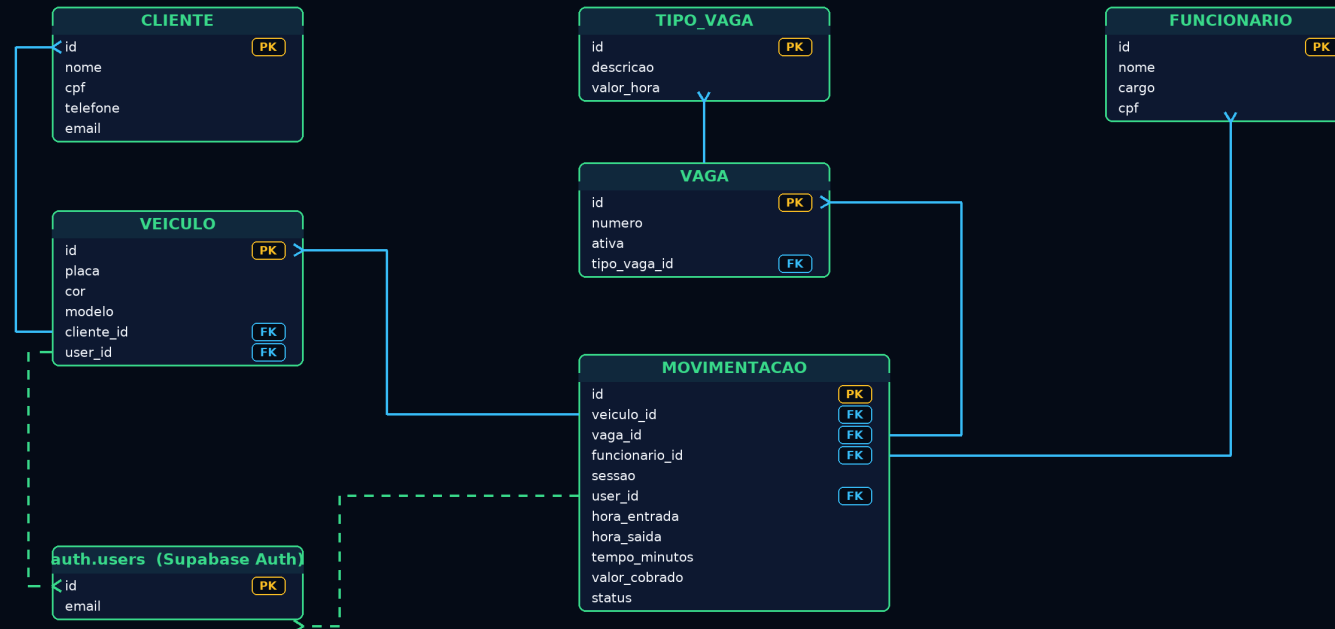
Relacionamento	Card.	Regra de negócio representada
CLIENTE — POSSUI — VEICULO	1:N	Um cliente possui vários veículos; cada veículo, um dono.
VEICULO — GERA — MOVIMENTACAO	1:N	Um veículo gera várias movimentações (histórico de entradas).
VAGA — OCUPA — MOVIMENTACAO	1:N	Uma vaga é ocupada em várias movimentações (uma por vez).
FUNCIONARIO — REGISTRA — MOVIMENTACAO	1:N	Um atendente registra várias movimentações.
TIPO_VAGA — CLASSIFICA — VAGA	1:N	Um tipo (tarifa) classifica várias vagas físicas.

No lado da MOVIMENTACAO a participação é total e unitária (1,1): cada movimento usa exatamente uma vaga, um veículo e um atendente.

Esquema relacional com chaves primárias e estrangeiras

Modelagem Lógica — Esquema Relacional (chaves primárias e estrangeiras)

PK = chave primária · FK = chave estrangeira · tracejado = vínculo com a conta (Supabase Auth)



Modelo lógico relacional: atributos atômicos, PK em todas as tabelas e FKs garantindo a integridade referencial (tracejado = vínculo com a conta em auth.users).

Notação relacional do modelo lógico

```
CLIENTE      ( id , nome, cpf, telefone, email )
VEICULO       ( id , placa, cor, modelo, cliente_id -> CLIENTE, user_id -> auth.users )
FUNCIONARIO   ( id , nome, cargo, cpf )
TIPO_VAGA     ( id , descricao, valor_hora )
VAGA          ( id , numero, ativa, tipo_vaga_id -> TIPO_VAGA )
MOVIMENTACAO ( id , veiculo_id -> VEICULO, vaga_id -> VAGA,
               funcionario_id -> FUNCIONARIO, sessao, user_id -> auth.users,
               hora_entrada, hora_saida, tempo_minutos, valor_cobrado, status )
```

id = chave primaria (sublinhado) | -> = chave estrangeira (FK)

Normalização aplicada (até a 3FN)

- 1FN — atributos atômicos e PK em todas as tabelas; sem grupos repetidos (um veículo com várias entradas = várias linhas de MOVIMENTACAO, nunca listas dentro de uma célula).
- 2FN — todas as tabelas têm PK simples (serial/bigserial); logo, não existe dependência parcial de chave composta.
- 3FN — nenhuma dependência transitiva: a tarifa depende apenas do TIPO_VAGA (vaga só guarda tipo_vaga_id); placa/cliente/vaga/atendente não se repetem em MOVIMENTACAO — entram por FK.
- Anomalias eliminadas: alteração de tarifa em um único lugar (TIPO_VAGA); inclusão de vaga sem movimentação; exclusão de movimentação não apaga cliente, vaga ou funcionário.
- tempo_minutos e valor_cobrado são derivados, mas armazenados como histórico auditável, calculados pelo banco (TRIGGER) — nunca digitados.

Integridade referencial (chaves estrangeiras)

FK (tabela.origem)	Referencia	Ação ON DELETE
veiculo.cliente_id	cliente.id	—
vaga.tipo_vaga_id	tipo_vaga.id	—
movimentacao.veiculo_id	veiculo.id	CASCADE
movimentacao.vaga_id	vaga.id	RESTRICT
movimentacao.funcionario_id	funcionario.id	RESTRICT
veiculo.user_id / movimentacao.user_id	auth.users.id	CASCADE

CHECKs complementares: hora_saida >= hora_entrada · status IN ('ESTACIONADO','SAIU') · valor_hora >= 0 · UNIQUE em placa, cpf e numero da vaga.

DDL — CREATE TABLE, TRIGGER, índices e RLS

```
CREATE TABLE movimentacao (  
    id                bigserial PRIMARY KEY,  
    veiculo_id        integer NOT NULL REFERENCES veiculo(id)      ON DELETE CASCADE,  
    vaga_id           integer NOT NULL REFERENCES vaga(id)          ON DELETE RESTRICT,  
    funcionario_id    integer NOT NULL REFERENCES funcionario(id)  ON DELETE RESTRICT,  
    sessao            integer NOT NULL DEFAULT 1,  
    user_id           uuid REFERENCES auth.users(id) ON DELETE CASCADE,  
    hora_entrada       timestampz NOT NULL DEFAULT now(),  
    hora_saida        timestampz,  
    tempo_minutos     integer,  
    valor_cobrado      numeric(8,2),  
    status             varchar(15) NOT NULL DEFAULT 'ESTACIONADO'  
                    CHECK (status IN ('ESTACIONADO','SAIU')),  
    CONSTRAINT chk_saida_apos_entrada  
        CHECK (hora_saida IS NULL OR hora_saida >= hora_entrada)  
);  
  
CREATE OR REPLACE FUNCTION fn_calcular_saida() RETURNS trigger AS $$ ... $$  
LANGUAGE plpgsql;    -- calcula tempo_minutos e valor_cobrado  
  
CREATE TRIGGER trg_calcular_saida BEFORE UPDATE ON movimentacao  
    FOR EACH ROW EXECUTE FUNCTION fn_calcular_saida();  
  
CREATE POLICY leitura_restrita_movimentacao ON movimentacao  
    FOR SELECT USING (auth.uid() IS NOT NULL AND user_id = auth.uid());  
  
-- Migração física (ALTER TABLE): vincula os dados a conta, sem quebrar nada  
ALTER TABLE movimentacao ADD COLUMN IF NOT EXISTS user_id uuid  
    REFERENCES auth.users(id) ON DELETE CASCADE;
```

DML — INSERT, UPDATE e DELETE (carga e operações)

```
-- Carga inicial (homologacao) - INSERT
INSERT INTO tipo_vaga (descricao, valor_hora) VALUES
    ('Comum', 6.00), ('Coberta', 9.00), ('PCD', 0.00), ('Idoso', 0.00);
INSERT INTO vaga (numero, tipo_vaga_id) VALUES
    (1, (SELECT id FROM tipo_vaga WHERE descricao = 'Comum')),
    (4, (SELECT id FROM tipo_vaga WHERE descricao = 'Coberta'));
INSERT INTO cliente (nome, cpf, telefone, email) VALUES
    ('Ana Beatriz Souza', '11111111111', '(31) 90000-0001', 'ana.souza@exemplo.com');
INSERT INTO movimentacao (veiculo_id, vaga_id, funcionario_id, sessao, user_id, status)
VALUES (<id>, <id>, <id>, 1, auth.uid(), 'ESTACIONADO');

-- UPDATE: saida do veiculo (a TRIGGER calcula tempo e valor no banco)
UPDATE movimentacao SET hora_saida = now(), status = 'SAIU'
WHERE veiculo_id = (SELECT id FROM veiculo WHERE placa = 'ABC1D23')
AND hora_saida IS NULL;

-- DELETE: limpeza dos dados da propria conta (executado pelo botao Reiniciar)
DELETE FROM veiculo WHERE user_id = auth.uid(); -- CASCADE limpa movimentacao
```

DQL — relatório com JOIN entre as 6 tabelas

```
-- Relatório completo: JOIN entre as 6 tabelas (painel administrativo)
SELECT m.id, v.placa, v.cor, c.nome AS cliente, vg.numero AS vaga,
       tv.descricao AS tipo_vaga, f.nome AS atendente,
       m.hora_entrada, m.hora_saida, m.tempo_minutos, m.valor_cobrado, m.status
FROM   movimentacao m
JOIN   veiculo      v  ON v.id   = m.veiculo_id
JOIN   cliente      c  ON c.id   = v.cliente_id
JOIN   vaga         vg ON vg.id  = m.vaga_id
JOIN   tipo_vaga    tv ON tv.id  = vg.tipo_vaga_id
JOIN   funcionario  f  ON f.id   = m.funcionario_id
WHERE  m.user_id = auth.uid()
ORDER BY m.hora_entrada DESC;
```

DQL — consultas estratégicas com agregações

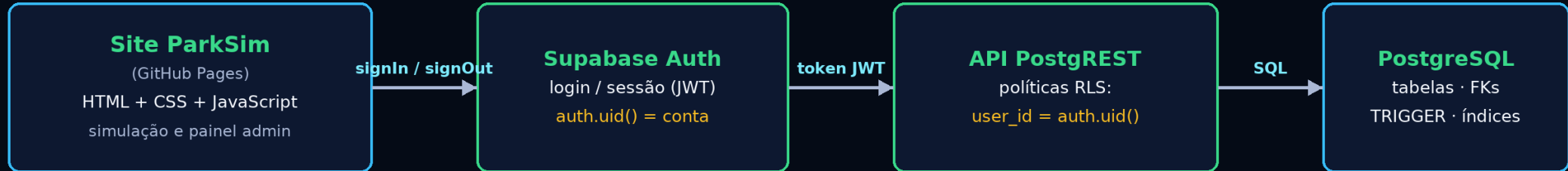
```
-- Faturamento por cliente no ultimo mes (exemplo do enunciado)
SELECT c.nome AS cliente, COUNT(m.id) AS total_movimentacoes,
       COALESCE(SUM(m.valor_cobrado), 0) AS faturamento_total
FROM cliente c
JOIN veiculo v ON v.cliente_id = c.id
JOIN movimentacao m ON m.veiculo_id = v.id
WHERE m.hora_saida >= now() - INTERVAL '1 month'
GROUP BY c.nome ORDER BY faturamento_total DESC;
-- Tambem: vaga mais utilizada (COUNT/GROUP BY), tempo medio por tipo (AVG),
-- funcionario que mais atendeu (COUNT) e veiculos estacionados agora.
```

Aplicação web e demonstração

- Site publicado: <https://lebot007.github.io/a3controlecarros/> · acesso admin de teste: admin@parksim.com / admin123.
- Fluxo: login (Supabase Auth) → simulação de entrada/saída → INSERT em veiculo e movimentacao → UPDATE de saída dispara a TRIGGER de cálculo → painel com SELECT JOIN em tempo real → exportação em PDF.
- Os registros podem ser inspecionados ao vivo no Table Editor do Supabase durante a demonstração.
- Persistência por conta: ao sair e entrar novamente na conta (ou recarregar a página), o relatório é restaurado — cada linha carrega o user_id do dono.
- Visitantes assistem à simulação sem gravar dados; o botão Reiniciar apaga somente os dados da conta logada.

Arquitetura da solução

Arquitetura da Solução — aplicação web + banco de dados em nuvem



INSERT / UPDATE / SELECT com JOIN — sempre com `user_id` da conta logada; visitante não grava dados

TRIGGER `trg_calcular_saida` calcula `tempo_minutos` e `valor_cobrado` no UPDATE

Front-end estático no GitHub Pages consome o Supabase: Auth (sessão JWT), API PostgREST com políticas RLS por conta e PostgreSQL com tabelas, FKs, TRIGGER e índices.

Segurança e persistência dos dados por conta

- Autenticação: Supabase Auth com sessão JWT; a conta logada é identificada por `auth.uid()`.
- Row Level Security: políticas de SELECT, INSERT, UPDATE e DELETE filtram por `user_id = auth.uid()` — uma conta nunca enxerga ou altera dados de outra.
- Migração física não destrutiva: `ALTER TABLE ... ADD COLUMN IF NOT EXISTS user_id + CREATE INDEX` + troca das políticas (executada com sucesso no SQL Editor do Supabase).
- Correção de comportamento: antes, recarregar a página apagava o banco inteiro; agora nada é apagado no carregamento e o histórico pertence à conta.
- Chave anon (publishable) apenas no front-end: toda proteção real fica nas políticas RLS do banco.

Conceitos de Banco de Dados aplicados

- Modelagem relacional: 6 tabelas normalizadas até a 3FN, ligadas por chaves estrangeiras;
- INSERT: criação de veículo (se a placa não existir) e da movimentação de entrada;
- UPDATE + TRIGGER: regra de negócio de cobrança executada dentro do banco;
- SELECT com JOIN e agregações (COUNT, SUM, AVG, GROUP BY) nos relatórios e no PDF;
- DELETE com ON DELETE CASCADE no ciclo de vida dos dados da conta;
- DDL completo (CREATE/ALTER TABLE, índices, trigger, políticas RLS) + DML + DQL em script .sql executável;
- Segurança e nuvem: Auth + RLS por conta e persistência real no PostgreSQL/Supabase.

Obrigado! Perguntas?

Integrante do Grupo 5	RA
João Vitor Alves Rodrigues	32513480
Rafael Luiz Ferreira de Souza	32511503
Pietro Cardoso de Oliveira	32515280
Gustavo Henrique Ramos	325141430
Pedro Henrique Martins	325130235

Site: <https://lebot007.github.io/a3controlecarros/> · Repositório: github.com/Lebot007/a3controlecarros

Entregas: documento de projeto (DOCX/PDF), apresentação (PPTX/PDF), diagramas e script .sql (DDL + DML + DQL).